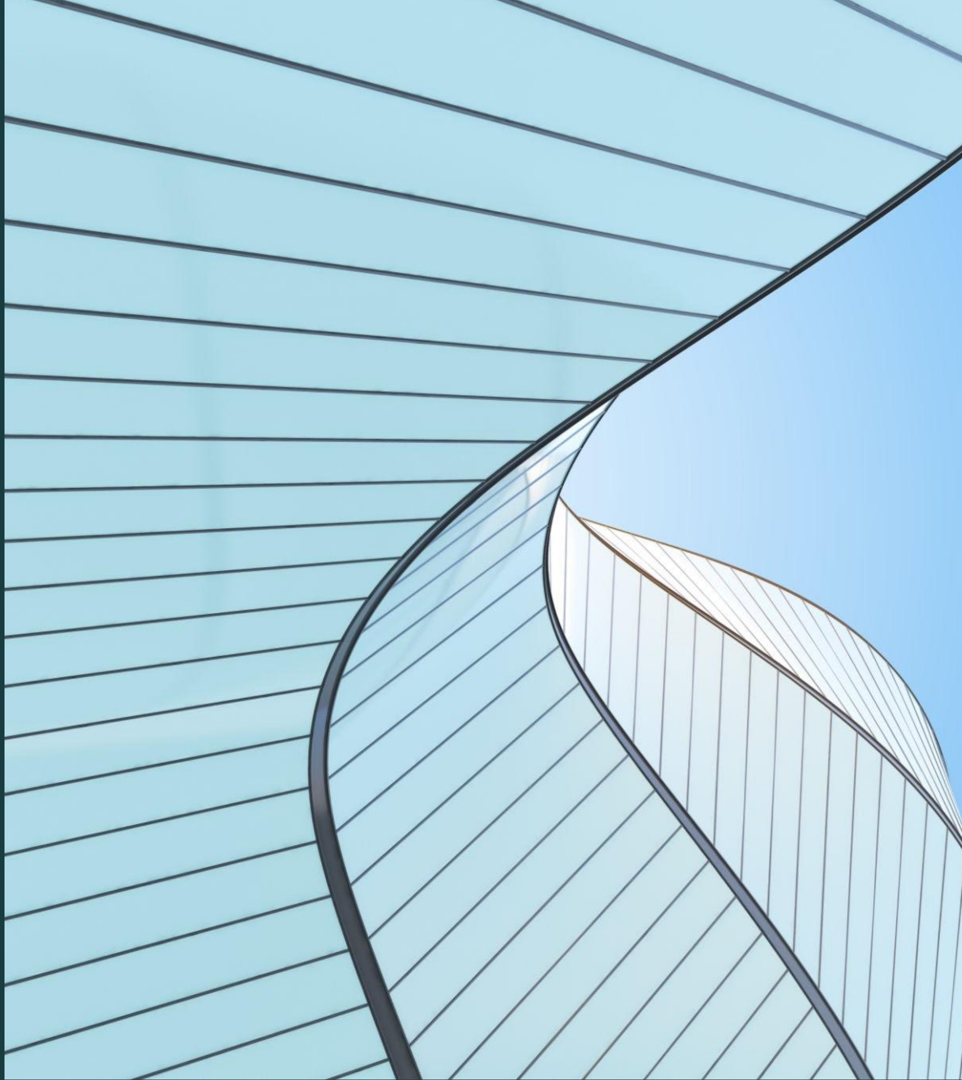
PolicyScope

A browser tool that helps people understand what they’re agreeing to

Amaan Bukhari, Emmanuel Garcia, Sebastian Lazarte, Mustafa Bayram, and Melvin Berkoh

Spring 2026 Capstone
Dr. Eljibari



Meet the Team

PROJECT MANAGEMENT

BACKEND

Project Manager / Architecture

Amaan Bukhari

Backend / Detection

Melvin Berkoh

Planning & Structure

Sebastian Lazarte

UI Design

Emmanuel Garcia

UI/UX Flow

Mustafa Bayram

Contents

What’s the Issue?

So what’s the problem?

- Terms of Service and Privacy Policies are extremely long
- Written in complex legal language
- Most users accept agreements without reading them
- Important clauses often go unnoticed

Result:

Users may unknowingly agree to data collection, automatic billing, or arbitration clauses.

Proposed Solution

So what should we do?

Well, here's where **PolicyScope** comes in.

Our solution is to create a browser extension that automatically analyzes policy pages while users browse.

Instead of requiring users to manually read through large documents, PolicyScope identifies key clauses and presents them in a simplified and organized format.

This allows users to quickly identify important sections such as billing policies, subscription conditions, data usage terms, and legal restrictions.

Scope

In Scope	**Out of Scope**
Clause detection of 8 various types of clauses (data collection and sharing, billing & subscriptions, legal & disputes, account & access, content & user rights, policy changes, and age restrictions)	Full legal interpretation
Real-time scanning of webpage text	Storing user browsing data
Highlighting detected clauses and providing summaries	Having biased opinions
Categorizing detected clauses	Supporting all browsers (focus on Chrome)

Stakeholders

01

Dr. Eljibari

Capstone Sponsor and Mentor

Provides project guidance and oversight

02

Royal Hansen

Potential Buyer/Platform Stakeholder

Represents organizations focuses on user privacy and policy transparency

03

Jason T.

NJIT Student (End User)

Represents student users interacting with online services

04

Rahat B.

Busy User/Parent (End User)

Represents everyday users with limited time to review policies

05

Amaan B.

Admin/System Operator

Responsible for system design and implementation

MDDDE (Process)

Manage

Project planning and timeline coordination. Regular Sunday / Tuesday team meetings

Define

Identified problem with unreadable online agreements. Defined key clause categories and MVP scope

Design

Planned browser extension workflow. Designed UI layout and detection structure

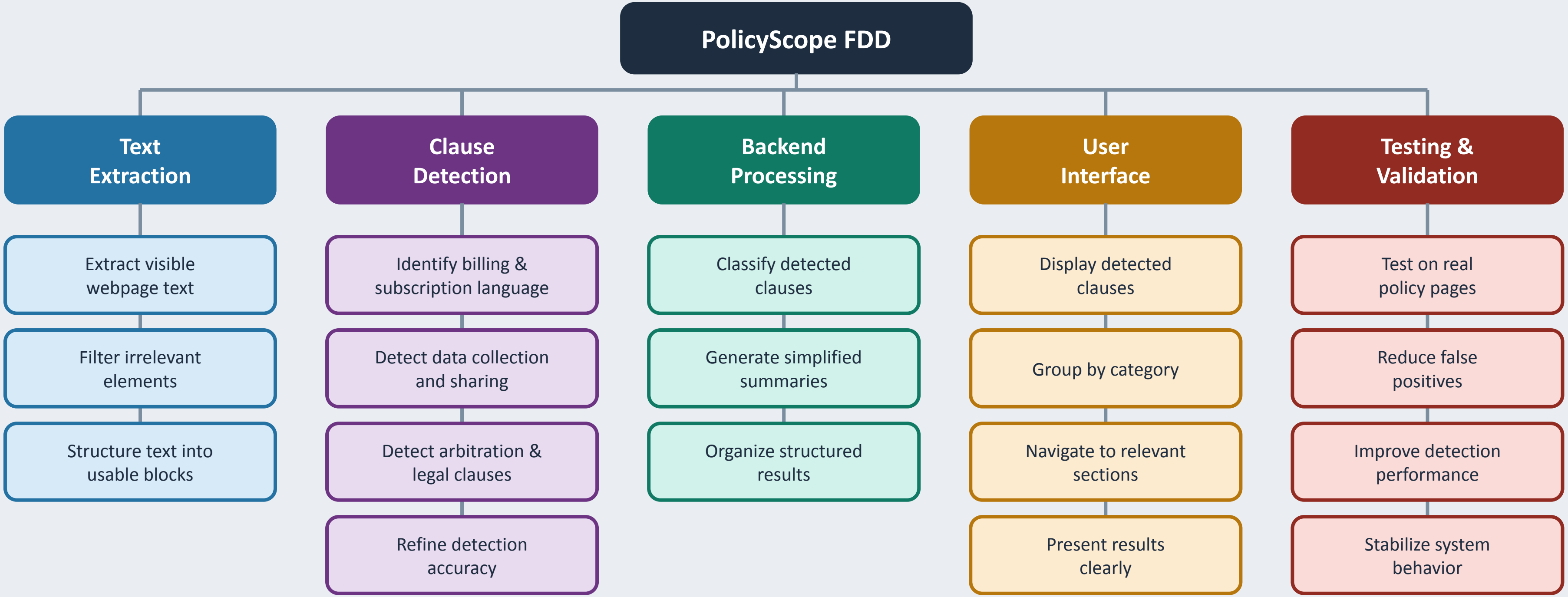
Develop

Built Chrome extension framework. Implemented webpage text extraction and clause detection

Evaluate

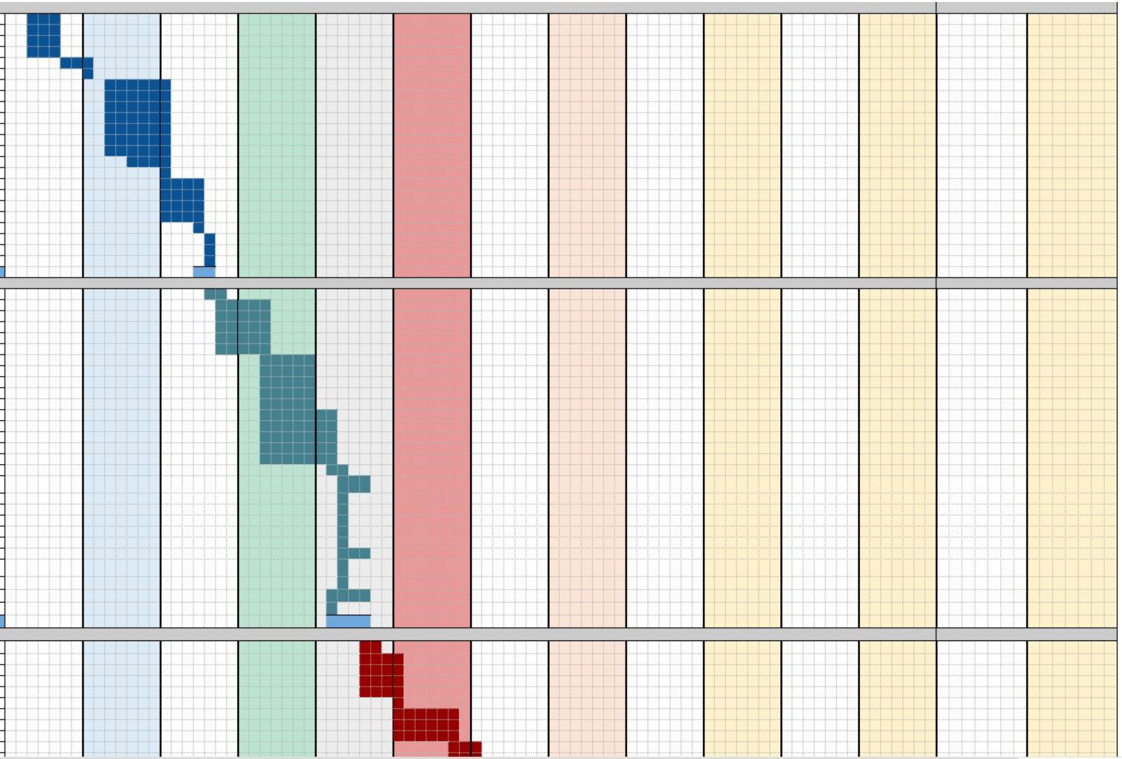
Tested on real policy pages. Refined detection logic and clause accuracy

FDD — PolicyScope Functional Design Document

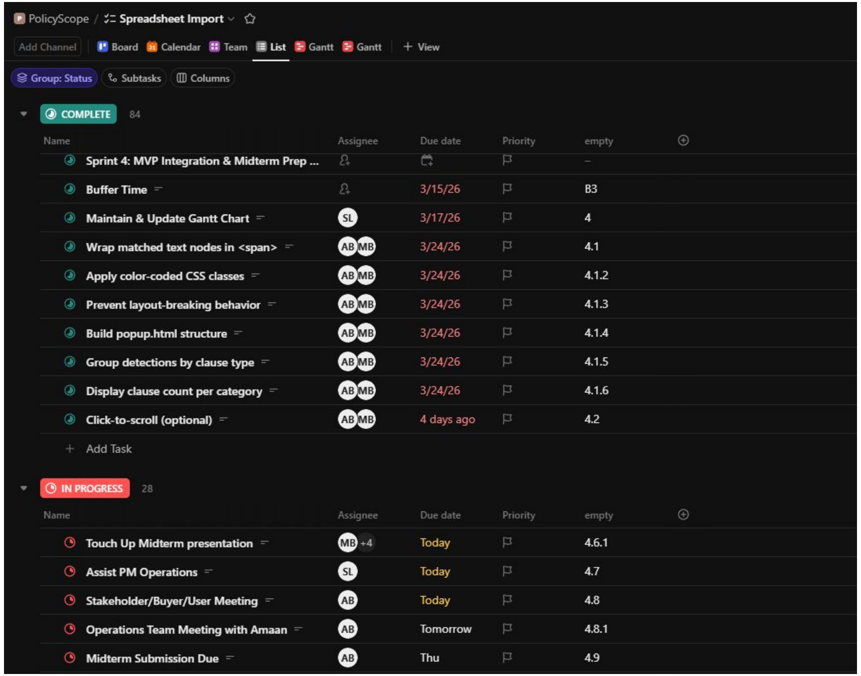


PolicyScope Ganntt Chart

	Sprint 1: Discovery and System Design due February 19, 2026						
1.1	Initial Policy Analysis & Clause Identification	Team	2/3/26	2/5/26	3	100%	N/A
1.1.1	Analyze Policy Structure & Page Layout Patterns	Team	2/3/26	2/5/26	3	100%	1.1.1 depends on 1.1
1.1.2	Define Initial Clause Categories	Team	2/3/26	2/5/26	3	100%	1.1.2 depends on 1.1.1
1.1.3	Initial UI Mockups & Detection Logic Concepts	Team	2/3/26	2/5/26	3	100%	1.1.3 depends on 1.1.2
1.1.4	Create Clause Example Dataset	Team	2/6/26	2/8/26	3	100%	1.1.4 depends on 1.1.2
1.1.5	Sunday Work Block	Team	2/8/26	2/8/26	1	100%	N/A
1.2	**Team Sync: Review Project Scope & Plan**	**Team**	**2/10/26**	**2/15/26**	6	100%	1.2 depends on 1.1.3
1.3	Confirm Team Roles & Responsibilities	Amaan	2/10/26	2/15/26	6	100%	1.3 depends on 1.2
1.3.1	Finalize Roles & Prepare Sprint 1 Tasks	Amaan	2/10/26	2/15/26	6	100%	1.3.1 depends on 1.3
1.4	Individual UI/Figma Assignment	Mustafa, Emmanuel	2/10/26	2/15/26	6	100%	1.4 depends on 1.3.1
1.5	Draft Dev Timeline to Trello	Melvin	2/10/26	2/15/26	6	100%	1.5 depends on 1.3.1
1.5.1	Draft Gantt Chart	Sebastian	2/10/26	2/15/26	6	100%	1.5.1 depends on 1.5
1.6	Chrome Extension Architecture Research	Amaan, Melvin	2/10/26	2/15/26	6	100%	1.6 depends on 1.3.1
1.6.1	Define System Architecture	Amaan, Melvin	2/12/26	2/15/26	4	100%	1.6.1 depends on 1.6
1.7	**Sunday Work Block: Finalize Submission**	**Team**	**2/15/26**	**2/15/26**	1	100%	N/A
1.8	Transfer Project Timeline to Task Management System	Amaan	2/15/26	2/18/26	4	100%	1.8 depends on 1.5.1
1.9	Finalize UI/Figma for Agreeance	Mustafa, Emmanuel	2/15/26	2/18/26	4	100%	1.9 depends on 1.4
1.9.1	Finalize Gantt Chart	Sebastian	2/15/26	2/18/26	4	100%	1.9.1 depends on 1.5.1
1.9.2	Final PM Approval of Scope & Schedule	Amaan	2/15/26	2/18/26	4	100%	1.9.2 depends on 1.9.1
1.9.3	**Final Scope & Timeline Review**	**Team**	**2/18/26**	**2/18/26**	1	100%	1.9.3 depends on 1.9.2
1.9.4	**Milestone: Scope & Time Mgmt Due**	**Team**	**2/19/26**	**2/19/26**	1	100%	1.9.4 depends on 1.9.3
1.9.6	**Operations Team Meeting with Amaan**	**Amaan**	**2/19/26**	**2/19/26**	1	100%	1.9.6 depends on 1.9.4
1.9.7	**Sprint Progress Report #1**	**Amaan**	**2/19/26**	**2/19/26**	4	100%	1.9.7 depends on 1.9.4
B1	BUFFER TIME		2/18/26	2/19/26	2		N/A
	Sprint 2: Detection Engine Development due March 5, 2026						
2	Maintain & Update Gantt Chart	Sebastian	2/19/26	2/20/26	2	100%	2 depends on 1.9.4
2.1	**Technical Foundation**	Melvin, Amaan	2/20/26	2/24/26	5	100%	2.1 depends on 2
2.1.1	Github Repositroy Setup	Melvin, Amaan	2/20/26	2/24/26	5	100%	2.1.1 depends on 2.1
2.1.2	Branch Structure Created	Melvin, Amaan	2/20/26	2/24/26	5	100%	2.1.2 depends on 2.1.1
2.3	Manifest.json Configured	Melvin	2/20/26	2/24/26	5	100%	2.2 depends on 2.1.2
2.3.1	Content Script Injection Test	Melvin	2/20/26	2/24/26	5	100%	2.2.1 depends on 2.2
2.4	**UI Direction Finalization**	Mustafa, Emmanuel	2/24/26	2/28/26	5	100%	2.3 depends on 2.2
2.4.1	Consolidate Previous Design Concepts	Mustafa, Emmanuel	2/24/26	2/28/26	5	100%	2.3.1 depends on 2.3
2.4.2	Develop Finalized Mockup V2	Mustafa, Emmanuel	2/24/26	2/28/26	5	100%	2.3.2 depends on 2.3.1
2.4.3	Refine UI Visual Direction (Structure & Layout Only)	Mustafa, Emmanuel	2/24/26	2/28/26	5	100%	2.3.3 depends on 2.3.2
2.4.4	Deliver Finalized Mockup V2	Mustafa, Emmanuel	2/24/26	2/28/26	4	100%	2.4 depends on 2.2
2.5	**Technical Research**	Melvin, Amaan	2/24/26	3/2/26	7	100%	2.4.1 depends on 2.4
2.5.1	Conduct Technical Implementation Research	Melvin	2/24/26	3/2/26	7	100%	2.4.2 depends on 2.4
2.5.2	Evaluate Pattern Detection Approaches	Melvin	2/24/26	3/2/26	7	100%	2.4.3 depends on 2.4
2.5.3	Assess Legal & Platform Constraints	Melvin	2/24/26	3/2/26	7	100%	2.4.4 depends on 2.4.1
2.5.4	Identify Tools & Networks	Melvin	2/24/26	3/2/26	7	100%	2.5 depends on 2.3.3
2.5.5	Deliver Technical Research Summary	Melvin	3/2/26	3/3/26	2	100%	2.5.1 depends on 2.5
2.6	**UI Implementation**	**Amaan, Sebastian, Mustafa, Melvin, Emmanuel**	**3/3/26**	**3/5/26**	1	100%	2.5.2 depends on 2.5.1
2.6.1	Convert Figma to HTML Structure	Mustafa, Emmanuel, Melvin	3/3/26	3/3/26	1	100%	2.5.3 depends on 2.5.2
2.6.2	Implement Base CSS Layout	Sebastian	3/3/26	3/3/26	1	100%	2.5.4 depends on 2.5.1
2.6.3	Implement Expandable Sections	Melvin, Amaan	3/3/26	3/3/26	1	100%	2.5.5 depends on 2.5.3
2.6.4	UI Logic Mapping (Categories → Visual Output)	Melvin, Amaan	3/3/26	3/3/26	1	100%	2.6 depends on 2.5
2.6.5	UI Review Sync (Tuesday Night)	Emmanuel, Mustafa, Sebastian	3/3/26	3/3/26	1	100%	2.7 depends on 2.6
2.7	Assist PM Operations	Sebastian	3/3/26	3/5/26	3	100%	2.8 depends on 2.7
2.8.1	**Capstone Deliverable Workshop**	**Amaan, Sebastian, Mustafa, Melvin, Emmanuel**	**3/3/26**	**3/3/26**	1	100%	2.9 depends on 2.7
2.9	**Stakeholder/Buyer/User Meeting**	**Amaan**	**3/3/26**	**3/3/26**	1	100%	2.9.1 depends on 2.8
2.9.1	**Operations Team Meeting with Amaan**	**Amaan**	**3/2/26**	**3/5/26**	4	100%	2.9.2 depends on 2.8
2.9.2	**Sprint Progress Report #2**	**Amaan**	**3/2/26**	**3/2/26**	1	100%	N/A
B2	Buffer Time		3/2/26	3/5/26	4		N/A
	Sprint 3: Browser Extension Integration due March 15, 2026						
3	**Maintain & Update Gantt Chart**	**Sebastian**	**3/5/26**	**3/6/26**	2	100%	3 depends on 2.9.2
3.1	Implement DOM TreeWalker	Melvin, Amaan	3/5/26	3/8/26	6	100%	3.1 depends on 3
3.2	Filter navigation/header/footer elements	Sebastian, Mustafa	3/5/26	3/8/26	6	100%	3.2 depends on 3.1
3.3	Enforce minimum character threshold	Emmanuel	3/5/26	3/8/26	6	100%	3.3 depends on 3.2
3.4	Return structured text blocks (text, node)	Melvin, Amaan	3/5/26	3/8/26	6	100%	3.4 depends on 3.1
3.5	Test on real-world sites	Sebastian, Mustafa, Emmanuel	3/8/26	3/8/26	1	100%	3.5 depends on 3.4
3.6	Implement strong/weak keyword system	Melvin, Amaan	3/8/26	3/13/26	6	100%	3.6 depends on 3.4
3.6.1	Rule trigger logic (≥1 strong OR ≥2 weak)	Melvin, Amaan	3/8/26	3/13/26	6	100%	3.6.1 depends on 3.6
3.7	Output structured detections (type, node, text)	Melvin, Amaan	3/8/26	3/13/26	6	100%	3.7 depends on 3.6.1
3.8	Assist PM Operations	Sebastian	3/13/26	3/15/26	3	100%	3.8 depends on 3.7



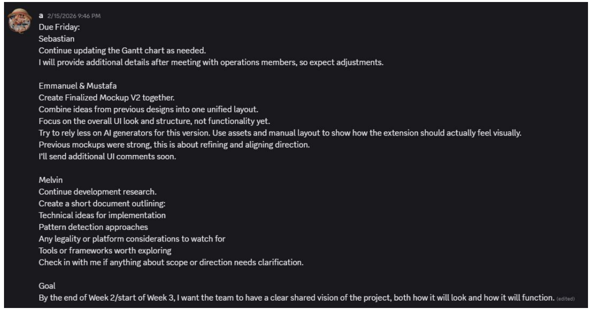
Scrum / Project Management



Clickup Project Management

Scrum Practices:

- Weekly progress reviews
- Iterative improvements to detection logic
- Continuous collaboration across development tasks



a 2/15/2026 9:46 PM
Due Friday:
Sebastian
Continue updating the Gantt chart as needed.
I will provide additional details after meeting with operations members, so expect adjustments.

Emmanuel & Mustafa
Create Finalized Mockup V2 together.
Combine ideas from previous designs into one unified layout.
Focus on the overall UI look and structure, not functionality yet.
Try to rely less on AI generators for this version. Use assets and manual layout to show how the extension should actually feel visually.
Previous mockups were strong, this is about refining and aligning direction.
I'll send additional UI comments soon.

Melvin
Continue development research.
Create a short document outlining:
Technical ideas for implementation
Pattern detection approaches
Any legality or platform considerations to watch for
Tools or frameworks worth exploring
Check in with me if anything about scope or direction needs clarification.

Goal
By the end of Week 2/start of Week 3, I want the team to have a clear shared vision of the project, both how it will look and how it will function. (edited)

Discord / Text Mini Assignment Structure

Technical Implementation

Modular Extension Architecture

- Separated core components: parsing, detection, highlighting, popup UI, backend summaries, and settings.
- Modular structure improves debugging, maintainability, and feature updates.

Secure Backend Processing

- AI summarization handled through a backend API.
- Prevents exposing API keys inside the browser extension.

Reliable Data Handling

- Input validation before processing policy text.
- Request sizes trimmed before sending to backend services.
- Runtime and API errors handled safely.

Safe UI Rendering

- Detected summaries rendered as plain text.
- Avoided unsafe DOM injection or HTML execution.

Development Process (SCRUM)

Incremental Development Approach

Development followed an iterative SCRUM-style workflow:

Stage 1 - Core Workflow

- Implemented text extraction and clause detection pipeline.

Stage 2 - User Interface

- Added popup interface and policy highlighting.

Stage 3 - Category Expansion

- Expanded clause detection categories and improved classification.

Stage 4 - Customization

- Added settings, user configuration options, and UI refinements.

Result

- Continuous testing and feedback-driven improvements.
- Working product maintained at each development stage.

QA Testing


TOTAL RESPONSES
24
Survey participants
EASE OF UNDERSTANDING TOS
4.5 / 5
Average across 24 respondents
HELPFULNESS FOR AGREEMENTS
4.6 / 5
Average across 24 respondents
EASE OF UNDERSTANDING — SCORE DISTRIBUTION
5 16 responses 66.7%
4 6 responses 25%
3 4.2%
2 0%
1 4.2%
HELPFULNESS FOR AGREEMENTS — SCORE DISTRIBUTION
5 18 responses 75%
4 5 responses 20.8%
3 0%
2 0%
1 4.2%
POLICY INFORMATION — WHAT MATTERS MOST
Data Collection 15
Billing & Subscriptions 14
User Rights 13
Data Sharing 12
Account & Access 8
Legal & Disputes 7
Policy Changes 7
Age Restrictions 4
WOULD RECOMMEND OR USE
96%
YES
Yes 23
Maybe 1
No 0
MOST USEFUL FEATURE
Simplified Explanations 15
Organized Summaries 5
Clause Detection 2
Color-Coded Categories 2

User Testing Results

Participants: 24 testers

Ease of Understanding Terms of Service

- **Average score:** 4.5 / 5

Helpfulness When Reviewing Agreements

- **Average score:** 4.6 / 5

Most Useful Features

- Simplified explanations
- Clause detection
- Organized summaries

Recommendation Rate

- Majority of testers indicated they would use or recommend PolicyScope.

Detection Categories

User Feedback and Improvements

Common suggestions from testers:

- Notifications when policies change
- More clause categories
- Improved UI highlighting
- Community voting on clause importance

Future Improvements

- Policy change alerts
- Expanded detection categories
- UI enhancements
- Cross-browser support

System Architecture

Webpage Terms of Service
↓
Text Extraction Module
↓
Content Filtering
↓
Clause Detection Engine
↓
Category Classification
↓
PolicyScope UI Display

Extension analyzes visible policy text.
Detection engine identifies key clauses.
Results presented to user in organized categories.

Challenges and Solutions

Challenges	Solutions
Understanding Policies: Understanding how policies are structured and written.	Researching common clause patterns and policy structures.
Team Coordination: Coordinating development across multiple contributors.	Discord, messaging, regular check-ins / meetings, clear role assignments
Time Management: Balancing development work with other academic responsibilities.	Using structured planning and dividing work across team members

Demo

Live demonstration of PolicyScope analyzing a policy page.

The demo will show:

- Webpage text analysis
- Detection of key policy clauses
- Categorized results presented to the user

Thank You! See you soon!